4.4 PPT

4.4 RAG

开篇

资料一多，按文件名查找就不够用了。

项目资料可能包括：

- 几百份文档
- 大量源码
- 多年故障记录

全部交给模型：成本高，无关内容也会干扰回答。

只挑几份交给模型：容易漏掉关键要求。

例如：修改位置、验收条件。

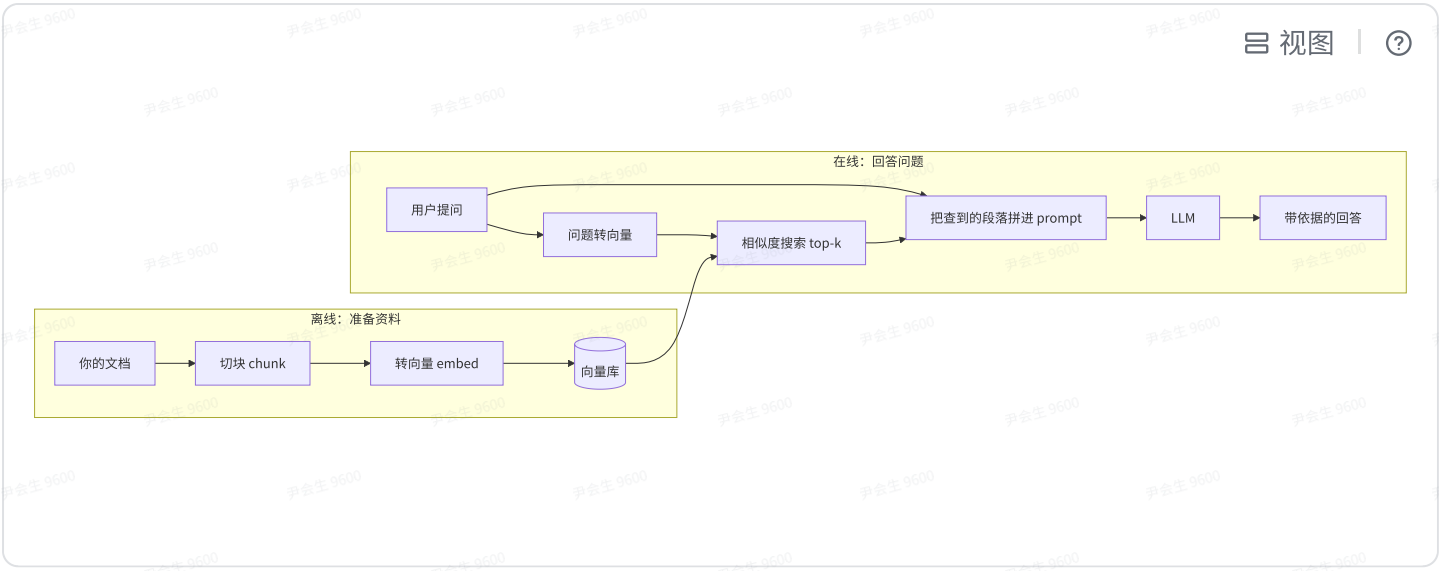
RAG 是什么

RAG = Retrieval-Augmented Generation，检索增强生成。

> 先从资料里找到相关证据，再把证据交给模型。

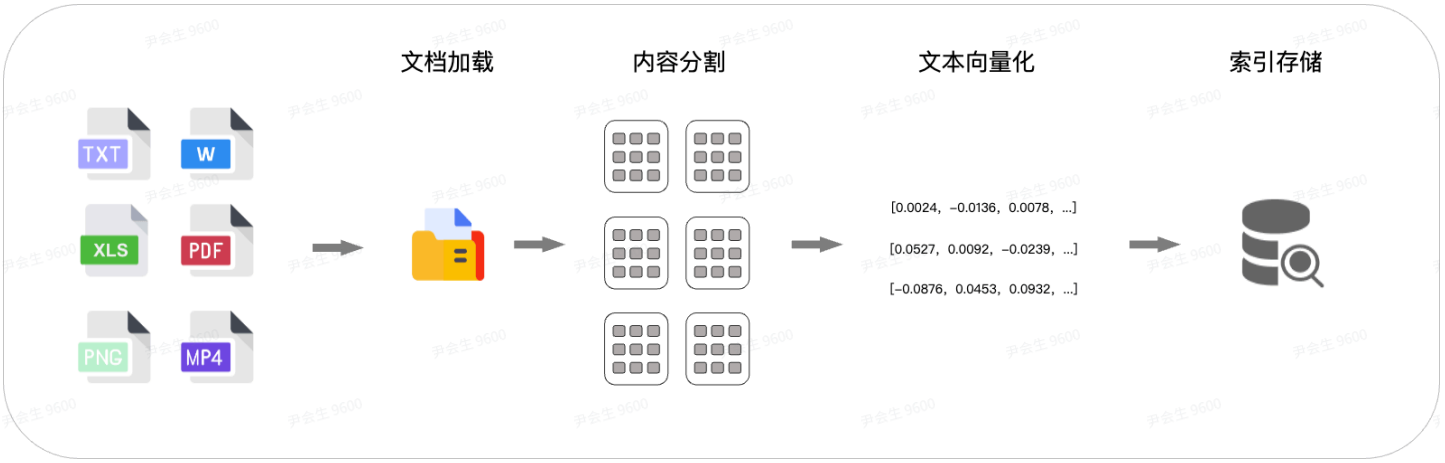
RAG 增强的是**当前请求的输入**，不是模型参数。

RAG 的两个阶段：



阶段一：建立索引

提前处理资料，避免每次提问都扫描全部文件。



四个步骤：

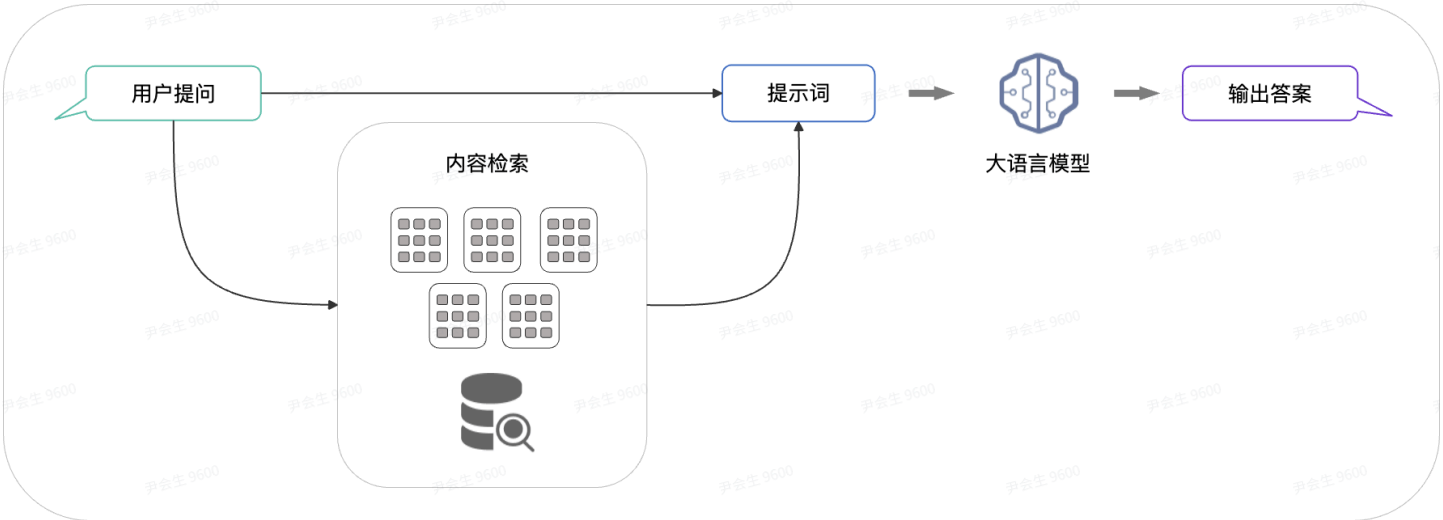
1. 文档解析
2. 文本分段
3. 文本向量化
4. 存储并建立索引

Embedding（文本向量化）：把文本转换成可计算相似度的数字向量。

程序将向量、原文和来源信息存入 Milvus。

Milvus：用于保存和检索向量的数据库。

阶段二：检索与生成



用户提问后：

1. 将问题转换成向量
2. 从索引中检索相关文本块
3. 筛选证据并加入模型输入
4. 模型根据证据生成回答
5. 标明答案来源

一句话记住 RAG

一句话记住：RAG 就是给 LLM 开卷考试。

模型要答得好，既要会分析，也要能找到正确的资料。

本节实践路线

继续围绕**登录会话项目**。

使用 **LlamaIndex** 组织文档读取、分块和检索。使用 **Milvus** 保存向量。

代码块

```
# demo.py
from pathlib import Path

# 跳过 NLTK 下载，避免网络问题导致的示例变慢或失败
```

```python
import os
os.environ["LLAMA_INDEX_DISABLE_NLTK"] = "1"

# 消除 WARNING 警告信息，避免干扰阅读；生产环境按需设置日志级别
import logging
logging.basicConfig(level=logging.ERROR)

from llama_index.core import Document, VectorStoreIndex

# 复用项目里已经接好的 MiniMax embedding 与 LLM
from rag.minimax import MiniMaxEmbedding, build_llm

print("正在解析文件...")
# 直接读取 corpus/docs 下的 Markdown 文档，加载为 Document 对象
docs_dir = Path(__file__).resolve().parent / "corpus" / "docs"
documents = [
    Document(
        text=path.read_text(encoding="utf-8"),
        metadata={"file_path": str(path.relative_to(docs_dir.parent.parent))},
    )
    for path in sorted(docs_dir.glob("*.md"))
]

print("正在创建索引...")
# from_documents 方法包含切片与建立索引步骤；embedding 用 MiniMax embo-01
index = VectorStoreIndex.from_documents(
    documents,
    embed_model=MiniMaxEmbedding(model_name="embo-01"),
)

print("正在创建提问引擎...")
query_engine = index.as_query_engine(
    # 设置为流式输出
    streaming=True,
    # LLM 用 MiniMax-Text-01（OpenAI 形状接口，api_base / key 已在 build_llm 里配好）
    llm=build_llm(),
)

print("正在生成回复...")
streaming_response = query_engine.query("登录会话的有效期是多久？到期边界应该怎么判定？")
print("回答是：")
# 采用流式输出
streaming_response.print_response_stream()
print()
```

```
49
```

按五步完成：

1. 用 Markdown 跑通最小闭环，观察证据怎样进入回答。
2. 完善解析和分块，减少信息丢失和关键内容被切断。
3. 改进召回，让需要的证据进入候选结果。
4. 重排、筛选和压缩，提高交给模型的证据质量。
5. 对比质量、成本和延迟，用同一组问题验证改动是否有效。

实践经验：先跑通闭环，再逐步优化。
每一步都用同一组问题验证，才能看出改动带来的实际效果。

先看第一步：用 Markdown 跑通最小闭环。

第一步：最小检索闭环

目标：先跑通完整链路，再优化效果。

读取 → 分块 → 入库 → 检索 → 组装 → 回答 → 校验

保留每个环节的输出，答错时逐步定位原因：

- 资料没有读进来？
- 关键段落没有检索到？
- 模型看到了证据，却仍然答错？

用两道题观察链路

题目	需要的信息	观察重点
简单题：登录会话的有效期是多少分钟？	一条明确规则	能否检索到直接答案
复杂题：增加退出登录功能，需要改哪些文件？验收标准是什么？	修改位置、验收要求	多条证据能否都被检索到

简单题检查基础检索；复杂题更容易暴露证据遗漏。

1. 读取文档，保留来源

Markdown 标题清楚，适合检查读取与分块。

PDF、HTML、表格放到第二步处理。

安装依赖并运行脚本：

```
代码块
1  pip install -r rag44/requirements.txt
2  python scripts/step1_min_loop.py
```

读取代码看两点

- 是否遍历语料目录
- 是否为每份资料保留路径和类型

路径用于追溯和引用。**类型**用于后续过滤。

```
代码块
1  EXT_TO_TYPE = {".md": "doc", ".py": "code", ".html": "web", ".csv": "table",
   ".pdf": "pdf"}
2
3  def load_markdown_and_code() -> List[Document]:
4      docs = []
5      for path in sorted(CORPUS.rglob("*")):
6          if path.suffix not in {".md", ".py"}:
7              continue
8          docs.append(_doc_from_path(path, path.read_text(encoding="utf-8")))
9      return docs
```

`_doc_from_path` 将文本包装成 LlamaIndex 的 `Document`，并补充 `file_path`、`doc_type` 等元数据。

运行结果

```
代码块
1  [解析] 读到 9 个文档（Markdown/代码/历史记录）
```

9 个对象来自：

- 4 份文档
- 3 个源码文件
- 2 份历史记录

经验：数量只能帮助发现漏读，不能证明正文完整。

还要抽查内容，确认项目规则和需求记录确实读入。

资料读进来后，下一步把整份文件切成可单独检索的片段。

2. 按标题分块，让结果定位到具体小节

用户问“会话有效期是多少”，需要找到“有效期”相关规则。

如果整份规则文件只算一个检索单位，续期方式和适用范围也可能一起被带入。

做法：先按 Markdown 标题切分；小节过长时，再按句子拆分。

代码块

```
# rag/ingest.py :180
parser = MarkdownNodeParser()
inner = SentenceSplitter(chunk_size=384, chunk_overlap=0)
for node in parser.get_nodes_from_documents([doc]):
    for sub in inner.get_nodes_from_documents([node]) or [node]:
        nodes.append(_decorate(sub, doc))
```

两层分块器各有分工：

- `MarkdownNodeParser`：按标题组织分块。
- `SentenceSplitter`：控制过长小节的长度。
- `_decorate`：为分块补充来源等元数据。

源码文件走单独的代码分支。**这段 Markdown 分块逻辑不等于代码 AST 分块。**

分块结果

代码块

```
[分块] 策略=按标题  得到 30 个节点
       app/auth.py                                  § (整文件)
       app/config.py                                § (整文件)
       app/session.py                               § (整文件)
       docs/faq.md                                  § 常见问题（FAQ）
       docs/faq.md                                  § 常见问题（FAQ） > 用户反馈"登录状态老是掉"
       docs/faq.md                                  § 常见问题（FAQ） > 能否把有效期改长一点
       … 其余 24 个
```

LlamaIndex 把分块对象称为 `Node`。

本次生成 30 个节点。每个节点都能追溯到文件或小节，检索后才能说明“找到的是哪段内容”。

分块大小的边界

`chunk_size=384` 按分块器的 Token 口径计数，不是 384 个汉字。

模型输入还包括问题、来源和提示词。**不能把分块大小直接当作整次请求的 Token 用量。**

经验：分块入库，只代表资料可以被检索。只有本轮检索选中并装配的内容，模型这次才能看到。

3. 向量化并检索，检查关键证据有没有出现

用户说“登出”，资料里可能写“退出登录”。

只按字面匹配，可能漏掉相关内容。

Embedding（向量化）：把问题和文本转换成向量，让程序按语义相似程度查找候选。

MiniMax `embo-01`，记录的向量维度为 1536。

适配层提供文档和查询两个向量化入口。课堂先看清入口，不展开 HTTP 请求的完整实现。

代码块

```python
# rag/minimax.py :50
class MiniMaxEmbedding(BaseEmbedding):
    batch_size: int = 8

    # _embed 里省掉的是拼请求体、发 HTTP、重试和计时的部分
    def _embed(self, texts, kind):
        # kind="db" 用于入库，"query" 用于查询，两边必须一致
        ...

```

```
    def _get_text_embedding(self, text):    # 单个文档
        return self._embed([text], "db")[0]

    def _get_text_embeddings(self, texts): # 批量文档
        return self._embed(texts, "db")

    def _get_query_embedding(self, query): # 查询
        return self._embed([query], "query")[0]
```

文档与查询必须使用相互匹配的模型和配置。

`db` 和 `query` 表示向量化用途，不要求传入相同字符串。

经验：向量维度相同，不代表向量可以直接比较。

将分块存入 Milvus

本例使用 Milvus Lite，将向量保存在本地数据库文件中。

代码重点看三处：存储位置、集合名称、向量维度。

代码块

```
# rag/ingest.py :350
store = MilvusVectorStore(
    uri=MILVUS_URI,                # milvus_login_demo.db
    collection_name=COLLECTION,    # login_demo_chunks
    dim=EMBED_DIM,                 # 1536，必须和 Embedding 输出维度一致
    overwrite=overwrite,
    similarity_metric="COSINE",
)
index = VectorStoreIndex(nodes,
storage_context=StorageContext.from_defaults(vector_store=store),
                         embed_model=embed_model)
```

`dim` 必须与 Embedding 实际返回的向量长度一致。

更换模型时，要重新确认维度，并重建对应的文档向量。

初始化时，也要确认 `overwrite` 是否会覆盖已有集合。

入库记录

代码块

```
[入库] collection=login_demo_chunks  dim=1536
       embedding 7 次请求 / 31 条文本 / 4.3s；入库共 5.0s
```

这些是本次运行记录，不能直接当作其他数据量或环境下的性能结论。

设置候选数量并检索

索引建好后，使用相同的 Embedding 配置检索。

本例先保留三个候选：

代码块

```
# scripts/step1_min_loop.py :182
retriever = index.as_retriever(similarity_top_k=3)
hits = retriever.retrieve(QUESTION)
```

简单题：答案证据进入候选

代码块

```
[q_easy] 登录会话的有效期是多少分钟?
[检索] top_k=3  耗时 505ms
       [1] score=0.8289  docs/session-rules.md § 登录会话有效期规则 > 不适用范围
       [2] score=0.8257  docs/session-rules.md § 登录会话有效期规则 > 续期
       [3] score=0.8141  docs/session-rules.md § 登录会话有效期规则 > 有效期长度
```

包含答案的“有效期长度”排在第三，但仍进入候选。

只要后续装配时保留这段证据，模型就有机会答对。

经验：检查召回，要看答案所需的证据是否出现；不能只看第一名或分数。

分数接近不能单独证明排序不稳定，也不能代表事实可信度。

复杂题：相关话题不等于证据齐全

代码块

```
[q_hard] 如果我要给 login_demo 增加退出登录功能，需要改哪些文件？验收标准是什么?
[检索] top_k=3  耗时 541ms
```

```
3          [1] score=0.9454  history/2026-10-06-logout-request.md § 需求讨论 2026-
   10-06：增加退出登录功能
4          [2] score=0.9396  docs/session-rules.md § 登录会话有效期规则 > 有效期长度
5          [3] score=0.9320  history/2026-10-06-logout-request.md § 需求讨论 2026-
   10-06：增加退出登录功能 > 背景
```

结果找到了“退出登录”相关内容，却漏掉回答所需的修改位置。

分数高，也补不上缺失证据。

下一步：检查候选怎样进入模型输入，以及证据不足时模型会怎样回答。

4. 证据筛选与上下文装配

检索返回节点和分数。

模型需要**证据文本**，也需要能追溯证据的**来源**。

先看筛选规则

代码块

```python
kept = [h for h in hits if h.score >= 0.30]
```

本次所有候选分数都高于 `0.30`，因此没有候选被删除。

“有效期长度”与修改位置无关，但仍被保留。

经验：分数阈值要用标注问题集校准。
本次阈值没有起到筛选作用，不能把同一个固定值套用到所有模型和语料。

再看上下文装配

给每条证据编号，并附上路径和标题。

代码块

```python
def assemble(hits, max_chars=None):
    parts = []
    for i, hit in enumerate(hits, 1):
        text = hit.node.text.strip()
        parts.append(f"[{i}] 来源：{label(hit)}\n{text}")
```

```
    context = "\n\n".join(parts)
    if max_chars is not None and len(context) > max_chars:
        context = context[:max_chars]
    return context
```

模型回答中的 `[3]`，对应本轮装配的第三条证据。

程序可以根据来源标签反查原文。

上下文超预算时

不能直接截断整段字符串。

截断可能丢掉规则后半句或来源信息，让证据失去原意。

经验：筛选或压缩后，重新检查关键条件和来源是否完整。

到这里，本轮交给模型的输入已经确定。接下来检查模型是否依据这些证据回答。

5. Citation：回答引用来源

模型可能用通用开发经验补齐资料里没有的信息。

提示词要说明三件事：

- 回答只能依据哪些证据
- 怎样标注证据来源
- 证据不足时如何回答

代码块

```
PROMPT_TEMPLATE = """你是 login_demo 项目的助手。
规则：
1. 只能依据下面的证据回答，不允许使用证据之外的知识。
2. 每条结论后面用 [编号] 标出它来自哪条证据。
3. 证据里没有写的内容，直接回答“证据里没有写”，不要猜。

证据：
{context}

问题：{question}
"""
```

提示词可以约束回答方式，**但是否遵守，还要检查模型输出。**

简单题：引用可以定位到正确证据

```
代码块
1  [回答]
2  登录会话的有效期是 30 分钟 [3]。
3  [用量] prompt=793 completion=11 耗时=1.356s
4  [引用] 编号=[3]  越界=无  判定=通过
5         [3] → docs/session-rules.md § 登录会话有效期规则 > 有效期长度
```

模型引用了第三条证据。

回答中的“30 分钟”与该小节一致，引用也能定位到原文。

复杂题：有引用，不代表有依据

模型建议修改“登录页面或导航栏”“JavaScript 文件”“会话管理模块”和“路由文件”，并为这些说法加上引用编号。

但回答没有列出需求记录中明确指出的三个文件。

这些说法可能符合其他项目的常见做法，却不能证明它们就是当前项目的修改范围。

原稿的编号检查仍判定通过，因为所有引用都在 `[1]` 到 `[3]` 范围内。

经验：编号检查只能确认引用指向本轮存在的证据，不能确认证据支持对应结论。

所以界面应显示“引用编号合法”，不能据此显示“答案通过”。

要判断回答是否有依据，还需要核对回答内容。

6. 校验回答内容，明确自动检查覆盖了什么

先实现一个便于观察的检查器。

它从回答中提取文件路径、常量名、数字与单位、命令和布尔值，再检查这些实体是否出现在对应证据里。

先看检查器能识别什么：

```
代码块
1  PATTERNS = [
```

```
    ("file",    re.compile(r"[A-Za-z_][\w/.-]*\.(?:py|md|html|csv|pdf|json)")),
    ("const",   re.compile(r"\b[A-Z][A-Z0-9_]{3,}\b")),
    ("number",  re.compile(r"\d+\s*(?:分钟|秒|小时|天)")),
    ("command", re.compile(r"python -m pytest[^\n，。；]*")),
    ("bool",    re.compile(r"\b(?:True|False)\b")),
]
```

这些正则只能识别特定写法，不能理解所有自然语言结论。

简单题中的“30 分钟”可以被提取，并在证据中找到。**这说明数字有字面依据，不代表整段回答都正确。**

复杂题里的“登录页面或导航栏”“路由文件”等说法不符合这些模式，会被跳过。即使一个数字有依据，也不能据此认定其他结论正确。

检查回执表示什么

代码块

```
[一致性] 有依据 1 个 ['30 分钟']
         无依据 0 个 []
```

这条回执表示：检查器提取到一个实体，并在证据中找到了它。

没有被检查到的结论，仍需人工核对。

区分检查状态

不能把“没检查到”直接写成“正确”。

代码块

```
if not citations:
    verdict = "无引用"
elif bad:
    verdict = "引用越界"
elif not supported and not unsupported:
    verdict = "无可核对实体"
elif unsupported:
    verdict = "存在无依据实体"
else:
    verdict = "通过"
```

最后的“通过”也只代表**本次实体检查通过**。

例如，“可以修改测试”和“不能修改测试”都包含“修改测试”，但含义相反。实体出现，只能提供核对线索，不能证明整条结论成立。

经验：复杂问题要把回答拆成结论，逐条核对证据是否支持。

还要检查是否覆盖问题要求。复杂题同时询问修改文件和验收标准，漏掉其中一项，答案仍不完整。

从跑通链路，转向提高质量

第一步跑通了端到端链路，也能根据输出定位问题。

简单题：证据进入候选，回答可追溯

“有效期长度”进入候选。

模型据此回答“30 分钟”，引用也对应到原文。

复杂题：三个环节暴露问题

- **召回遗漏**：修改位置没有进入候选。
- **生成越界**：证据不足时，模型仍给出修改建议。
- **校验有限**：编号和实体检查没有识别这些建议缺乏支持。

三个问题要分别处理：

1. 让正确证据进入候选。
2. 让模型依据证据回答。
3. 检查回答结论是否得到证据支持。

经验：提示词不能补回没检索到的资料；引用编号也不能证明答案正确。

下一步，回到数据准备，检查“修改位置”和验收要求是否完整保留、切分是否合适、来源是否清楚。

确认入库内容没有问题后，再改进检索，让关键证据进入模型输入。

第二步：完善数据准备

第一步跑通了检索和回答。

但用户问“增加退出登录功能，需要改哪些文件、怎样验收”时，关键证据没有进入候选。

先不急着更换检索方法。先检查三件事：

- 相关资料有没有读进来？
- 解析时有没有丢掉规则？
- 分块时有没有把完整信息切开？

检索只能查询已经准备好的内容。

文件没加载，或“恰好到期”和“判定无效”被切开后失去关联，单纯调高 Top-K 也不能直接解决。

这一阶段继续使用登录会话项目，运行数据准备脚本：

```
代码块
1   python ./scripts/step2_data_prep.py
```

运行前，先明确检查顺序。脚本会依次展示五项结果：

1. 格式解析
2. 文档元数据
3. 分块对比
4. Overlap
5. 节点元数据

接下来沿着这五项输出，检查资料怎样从文件变成可检索的分块。

1. 文档解析：PDF、HTML、表格与结构化文档解析

资料不只有 Markdown 和源码。

这一步加入 PDF、HTML 和 CSV。

解析不仅要提取文字，还要保留标题、条件和值之间的关系。

为什么要保留关系

PDF 中有一张边界测试表：

情况	期望
到期前	True
恰好到期	False
到期后	False

如果解析结果变成“到期前、恰好到期、到期后、True、False、False”，条件和值就无法可靠对应。

即使命中“恰好到期”，也不一定能判断它对应哪个结果。

经验：加载器返回数据，只能说明程序拿到了内容；还要抽查解析结果是否保留了关键关系。

不同格式，要检查不同关系：

资料	必须保留的内容	抽查重点
文本 PDF	正文、章节、页码	边界规则是否完整，能否回到原页
扫描 PDF	OCR 文本、对应页面	<、<=、否定词是否识别正确
HTML	正文、标题、代码块、原 URL	导航是否混入正文，测试命令是否丢失
表格	表头、行列关系、单位、表名	“恰好到期”是否仍对应 False
JSON 等结构化资料	字段路径、对象关系、值	expires_at 属于哪个对象

加载器成功返回，只能说明程序拿到了数据。解析是否正确，还要抽查关键内容。

PDF：检查文字是否正确抽取

演示使用 `pypdf` 逐页提取文本。

重点看两点：文字来自哪一页；页面没有提取到文字时，程序如何处理。

代码块

```python
# ./scripts/step2_data_prep.py

def load_pdf(path: Path) -> str:
    reader = PdfReader(str(path))
    pages = []
    for i, page in enumerate(reader.pages, 1):
```

```
            text = (page.extract_text() or "").strip()
            pages.append(f"<!-- page {i} -->\n{text}")
        return "\n\n".join(pages)
```

实际回执：

```
代码块
[PDF ] 原始抽取 488 字符，开头：<!-- page 1 --> / 
        login_demo 产品需求说明 v2.5.0（草稿） / …
[PDF ] 清洗后 472 字符，开头：
        login_demo 产品需求说明 v2.5.0（草稿） / …
```

本次抽取到了文本，但这不能证明页面内容完整。

还要对照原页，检查边界规则、表格和符号。扫描件需要 OCR；复杂版式也可能需要专门解析。

代码里的页码注释只是临时标记。如果清洗时删除注释，要先把页码保存到页面或分块的元数据中，否则只能引用文件，无法准确定位页面。

HTML：保留标题，方便按小节分块

HTML 正文混在标签中。

先抽取标题和段落，再把标题转换成 Markdown 格式，供后续按标题分块使用。

```
代码块
def load_html(path: Path) -> str:
    soup = BeautifulSoup(path.read_text(encoding="utf-8"), "lxml")
    lines = []
    for tag in soup.find_all(["h1", "h2", "h3", "p", "li"]):
        text = tag.get_text(" ", strip=True)
        if not text:
            continue
        if tag.name.startswith("h"):
            lines.append(f"{'#' * int(tag.name[1])} {text}")
        else:
            lines.append(text)
    return "\n\n".join(lines)
```

关键是将 HTML 标题还原成 `#`、`##` 形式，后续才能复用按标题切分的逻辑。

实际回执：

```
代码块
[HTML] 原文含标签 532 字符
[HTML] 解析后 272 字符，开头：
        # login_demo v2.4.0 发布说明
        发布日期：2026-09-20。本次发布调整了登录会话的到期判定。
        ## 变更内容
        到期时刻改为判定为无效，…
```

本次结果保留了正文和标题层级。

字符数从 532 降到 272，只说明解析后文本更短，不代表被删掉的内容都无用。还要对照原页面，检查测试命令、代码块和关键限制是否保留。

发布说明描述某个版本的变更。判断当前代码是否已修复，还要核对源码和测试，不能把发布说明当成当前运行结果。

CSV：让每个值保留列名

CSV 有行列结构。

为了让每行可以独立检索，演示将列名和值组合成文本。重点检查：值是否仍绑定原来的列名。

```
代码块
def load_table(path: Path) -> str:
    rows = list(csv.DictReader(io.StringIO(path.read_text(encoding="utf-8"))))
    return "\n".join(
        "; ".join(f"{k}是{v}" for k, v in row.items() if v) + "。" for row in
rows
    )
```

实际回执：

```
代码块
[CSV ] 原文 5 行，第 1 行：
服务,会话类型,有效期,可续期,到期边界处理,负责人

[CSV ] 解析后 4 句，第 1 句：
服务是login_demo；会话类型是登录会话；有效期是30 分钟；
可续期是是；到期边界处理是恰好到期视为无效；负责人是认证组。
```

原文件有一行表头和四行数据，解析后得到四条记录。

“30 分钟”保留了“有效期”列名，因此单独检索到这段文本时，仍能看出它指登录会话的有效期。

当前格式会产生“可续期是是”这类不自然表达。可以改成“可续期：是”或“可续期：否”。**但本次实际运行还没有输出这种格式。**

表中还包含 `pay_gateway` 的支付会话。“五分钟”可能干扰登录会话问题。后续检索除了匹配“有效期”，还要检查服务范围。

正文中出现“服务是 `login_demo`”，不等于程序已经为分块添加了 `service` 元数据。**正文内容和可用于过滤的元数据，是两回事。**

到这里，三种新增格式都生成了文本。下一步检查程序为这些文档添加了哪些标签。

2. 文档清洗、元数据提取

清洗有一条红线：

可以去噪声，不能去限定词。

清洗时丢失的信息，后面的模型补不回来。

代码块

```
def clean_text(text: str) -> str:
    text = re.sub(r"<!--\s*page\s*\d+\s*-->", "", text)    # 去掉页码注释
    text = text.replace("\u3000", " ")                     # 全角空格换半角
    lines = [line.rstrip() for line in text.splitlines()] # 去行尾空格
    text = "\n".join(lines)
    text = re.sub(r"\n{3,}", "\n\n", text)                 # 连续空行压成一个
    return text.strip()
```

去掉页码注释前，要先确认页码信息已保存到元数据。否则，清洗会让来源失去页级定位。

元数据除了项目、版本和来源，还要区分资料性质；至少也要让路径能区分资料用途：

- `docs/`：项目规范
- 测试回执：某次执行结果
- `history/`：历史记录
-

用 YAML 保存文档元数据

Markdown 顶部可用 YAML 头信息记录元数据。解析时读入标签，正文中不保留头信息。

代码块

```
---
service: login_demo
topic: session-expiry
owner: 认证组
updated: 2026-10-01
---
```

代码块

```
def parse_front_matter(text: str) -> tuple[dict, str]:
    if not text.startswith("---"):
        return {}, text
    ...
    meta[key.strip()] = value.strip()
    body = "\n".join(parts[end + 1:]).lstrip("\n")
    return meta, body
```

当前读取结果

代码块

```
② 元数据：每个文档带上什么标签
========================================================================
共 12 个文档
  app/auth.py                          doc_type=code   source=py
  app/config.py                        doc_type=code   source=py
  app/session.py                       doc_type=code   source=py
  docs/faq.md                          doc_type=doc    source=md
  docs/payment-session.md              doc_type=doc    source=md
  docs/session-rules.md                doc_type=doc    source=md
  docs/testing-guide.md                doc_type=doc    source=md
  history/2026-09-12-expiry-bug.md     doc_type=doc    source=md
  history/2026-10-06-logout-request.md doc_type=doc    source=md
  pdf/PRD-login-2.5.0.pdf              doc_type=pdf    source=pdf
  tables/session-matrix.csv            doc_type=table  source=csv
  web/release-notes-2.4.0.html         doc_type=web    source=html
```

先区分两个字段：

- `doc_type`：资料类型
- `source`：文件格式

这两个字段不能直接说明资料是否可信、是否仍然有效。

例如，规范和历史记录在清单中都是 `doc_type=doc`、`source=md`，但用途不同：

- 规则文档：说明项目要求怎样实现。
- 历史记录：说明过去发生过什么。
- 测试回执：说明某次执行得到什么结果。

历史修复通过，不能证明当前代码仍然通过；PDF 草稿提出要求，也不能证明要求已经实现。

因此，读取时要保留资料的用途和状态。后续可以增加记录性质、版本、验证来源等字段。**当前回执没有展示 `record_type` 或 `version`，不能把这些说成已经实现。**

清洗也要遵守同一原则：保留“草稿”“当前版本”“不得”“仅限”等限定信息。它们决定一条记录何时适用、适用于谁。

资料和标签确认后，下一步检查分块是否保留了回答所需的完整内容。

3. 分块（Chunking）对比：完整性和粒度都要看

分块决定检索的**最小单位**。

- 单位太大：节点混有多个话题，模型还要自己挑内容。
- 单位太小：一句话被切开，条件和结果可能分离。

下面在同一批文档上对比六种切法：

策略（脚本标签）	节点数	平均字符	最长字符	带标题	函数完整
固定长度 96 Token，无重叠	71	85	277	19/71	6/10
固定长度 96 Token，重叠 24	78	89	259	19/78	6/10
按段落 ≤200 字符	89	67	383	32/89	8/10
按标题	36	168	2460	34/36	10/10
语义分块	15	407	1787	7/15	9/10
代码 AST	14	180	383	14/14	10/10

固定长度：增加重叠，不等于函数更完整

无重叠和重叠 24 都是 `6/10`。

本次实验中，增加重叠没有提高完整包含的函数数量。

经验：不能只看配置里启用了 Overlap，要检查实际分块结果。

按标题：内容完整，粒度可能太粗

按标题的函数完整度是 `10/10`，但最长节点达到 2460 个字符。

下一行输出显示，这个节点来自 `app/auth.py`，标题是“整文件”。

源码整份保留，函数自然没有被切开；但检索具体函数时，单位可能太大。

内容完整，不等于检索粒度合适。

AST：保留函数结构，提供更细的源码单位

AST 将三个源码文件切成 14 个节点，10 个函数都完整包含，平均长度为 180 个字符。

它按语法结构切分，保留函数边界，检索单位也比整文件更细。

但 AST 识别的是语法结构，不是调用链。找到会话判断函数后，仍可能需要补读调用者、会话存储和配置。

注意：AST 这行只统计源码，不能与覆盖全部资料的方案直接比较节点总数。

检查标签和实际输出是否一致

“按段落 ≤200 字符”的最长节点是 383 字符。

脚本标签里的 200，不是本次结果的严格上限。需要检查超长段落是否经过二次拆分，不能直接向学员保证每个分块都不超过 200 字符。

“带标题”数量少，也不等于来源丢失。节点可能仍保留文件路径，只是没有小节位置；带标题多，也不代表节点包含了正确答案。

下一轮分块方案

这张表展示的是分块结构，不是召回效果排名。

下一步可先采用：

- 规则和需求记录：利用标题结构切分
- 源码：利用 AST 保留函数边界

再用同一组问题检索，验证关键证据是否更容易找到。

4. Chunk Overlap

Overlap（分块重叠）让相邻分块重复一部分内容，减少关键句刚好被切开的情况。

先看一个具体例子。

从 `session-rules.md` 取出“到期边界”一节，共 132 个字符，窗口设为 40：

```
代码块
1  [无重叠 overlap=0 ] 切出 4 块，第 1 块结尾：'当前时刻恰好等于到期时刻时，会话必须判定为
   无效。'
2                      含“不要写成 `now <= expires_at`”? 否
3  [有重叠 overlap=15] 切出 6 块
4                      含“不要写成 `now <= expires_at`”的块是第 5 块：'s_at`，不要写
   成 `now <= expires_at`。\n'
```

加入重叠后，目标句出现在一个分块中，说明它缓解了这个例子里的边界问题。

但第 5 块仍以 `s_at` 开头，不是完整的语义段落。**包含目标句，不代表上下文和条件都完整。** 检索到这个分块后，还要检查它是否足以回答问题。

重叠也会增加重复内容

接着，脚本改用 `docs/session-rules.md` 的真实正文，共 366 个字符，不含 front matter：

```
代码块
1  换到真实的 docs/session-rules.md 正文（366 字符，已去掉 front-matter）：
2    overlap=0  切成 7 块，块数 × 窗口 = 420 字符
3    overlap=20 切成 10 块，块数 × 窗口 = 600 字符
```

本次设置下，分块数从 7 增至 10。重叠通常会增加重复文本，后续要关注 Embedding 用量和检索结果去重。

但 `块数 × 窗口` 只是粗略估算，不是实际存储字符数，也不是数据库占用。**不能据此得出存储量增加了 43%。**

要测实际代价，需要统计：

- 所有分块的实际长度
- Embedding 用量
- 索引大小

调整 Overlap 的顺序

1. 找出被切断的规则。
2. 增加适量重叠。
3. 检查关键内容是否完整、重复是否过多。

经验：不要照搬固定比例，也不要把字符演示中的 15、20 直接套用到 Token 分块参数。

5. Chunk Metadata：每个分块保留什么信息

节点上的字段，决定后续能否过滤、引用和定位原文。

代码块

```
{
  "id": "4c670033-7c1c-40d7-8cde-c5fbf258fac3",
  "metadata": {
    "file_path": "docs/testing-guide.md",
    "doc_type": "doc",
    "source": "md",
    "service": "login_demo",
    "topic": "testing",
    "owner": "认证组",
    "updated": "2026-09-20",
    "heading": "login_demo 测试指南 > 测试入口",
    "header_path": "/login_demo 测试指南/"
  },
  "text_chars": 173
}
```

这个节点包含项目、主题、维护人、更新时间，以及文件和标题位置。

字段	用途
service	限定项目范围
topic	标记主题
file_path	定位源文件
heading、header_path	定位小节
id	供程序反查节点
owner、updated	提供维护信息

`owner` 和 `updated` 不能单独证明正文正确或仍然适用。

当前输出没有 `record_type`、源码版本或验证回执。因此，不能说程序已经用这些字段区分规范、历史记录与验证结果；也不能根据一个节点推断全部 12 份文档都有相同标签。

代码节点的定位信息

代码块

```
{
  "metadata": {
    "file_path": "app/auth.py",
    "doc_type": "code",
    "source": "py",
    "heading": "(模块变量)",
    "kind": "assign",
    "lines": "16-16"
  },
  "text": "_TOKEN_STORE: dict[str, dict] = {}"
}
```

这个 AST 节点保留了模块变量，并标明它来自 `app/auth.py` 第 16 行。

行号帮助回到源码查看，不代表这里就是需要修改的位置。模型还要结合问题和当前源码判断；源码变化后，也要重新核对位置。

从数据准备进入检索优化

第二步确认了：

- 12 份资料进入加载结果。

- 新增格式已经产生文本输出。
- 可以比较不同分块方案的完整性和粒度。
- 示例节点能通过路径、标题或行号定位原文。

同时，还要继续检查：

- PDF 清洗后是否保留页码定位。
- 表格分块是否带有项目标签。
- 超长段落是否经过处理。
- 是否区分资料性质和版本。

本次运行没有展示召回率、答案正确率或 Milvus 入库结果，因此只能说明部分数据准备流程已验证，不能据此断定检索质量已经提高。

进入第三步前，先确认退出登录的修改位置和验收要求完整存在于待索引分块中，再检查过滤字段是否覆盖相关资料。然后建立或更新索引，用同一组问题比较向量检索、关键词检索和混合检索，观察关键证据能否进入候选。

第三步：检索（解决召回问题）

第一步，用户问“退出登录需要改哪些文件”，程序找到了需求背景，却没找到修改清单。

模型没有拿到关键资料，回答就缺少依据。

这一步先改进检索，让需要的资料进入结果，再交给模型回答。

实验范围

继续使用登录会话项目，运行检索脚本：

```
代码块
1   python scripts/step3_retrieval.py
```

本次实验固定使用：

- 9 份 Markdown 和源码

- 30 个分块
- 8 道题

这样可以比较不同检索配置。

第二步新增的 PDF、HTML、CSV 和 AST 分块，暂时不在本组实验中。

先看懂三个指标

每道题都标注了正确证据，代码中称为 `gold`。

例如，“用哪条命令验证”对应测试指南中的“测试入口”。

Top-3 表示最终保留排名前三的结果。结果中包含标注的文件和小节，就记为找到该证据。

指标	看什么
召回率	每题找回了多少标注证据，再对八题取平均
命中率	八道题中，有多少题至少找到一条标注证据
完全命中题数	有多少题找齐了全部标注证据

一道题标注了两处证据：

- 找到一处：召回率为 `0.5`，这题算命中。
- 找到两处：召回率为 `1`，这题才算完全命中。

本步检查的是证据位置，不是最终答案。命中不等于答对。

1. 先测两路检索，看看各漏了多少

向量检索按语义找资料，BM25 按词匹配。

先分别运行两种检索，再比较结果。

配置	召回率	命中率	完全命中
向量	0.312	0.375	2/8
BM25	0.625	0.75	4/8

当前这批问题中，向量检索找齐了 2 道题的证据，BM25 找齐了 4 道。

经验：关键词匹配补到了向量检索漏掉的部分资料。

接下来更换 Embedding 模型，看看向量检索能否找得更准。

2. Embedding：换模型后，重新建库再比较

Embedding 把问题和分块转换成向量，再由程序比较它们的相似程度。

模型不同，检索结果也可能不同。

本次保持分块和问题不变，只比较三种向量配置：

配置	维度	召回率	完全命中	建库及八题评估耗时
MiniMax embo-01	1536	0.312	2/8	9.7 秒
MiniMax 截断	256	0.312	2/8	9.8 秒
BGE-small-zh 本地	512	0.688	2/8	2.1 秒

BGE 的召回率从 `0.312` 提升到 `0.688`，完全命中的题目从 2 道增加到 5 道。

因此，后续实验使用 BGE 作为向量模型。

经验：BGE 更适合本次问题集，不代表它在其他项目中一定更好。

256 维截断后的汇总指标没有变化，只能说明本次实验没测出差异，不能说明其他维度没有作用。维度更高，也不代表检索一定更准。

正确解读耗时

表中时间看起来标注为“建库秒”，但实际计时包含建库和八道题的检索评估。

本次 BGE 总耗时更短，不能把这个差异全部归因于建库速度。

更换模型后重新建库

Embedding 模型更换后，要重新生成文档向量。

文档和问题必须使用匹配的模型及处理方式，否则两边的向量无法正确比较。

3. Vector Database：Milvus 建库和查库分开

前面一直在使用 Milvus，这里把它单独讲清楚。

大纲列出 FAISS、Milvus、Chroma，要求至少实操一种。本节只安装 Milvus，因为三者的定位不同，不需要为了横向比较同时安装三套。

- **FAISS**：向量搜索库，擅长快速查找最近的 `k` 个向量。持久化、并发和元数据过滤等能力，需要应用另外配套。
- **Chroma**：集成式向量存储，上手快，适合原型和中小规模项目。
- **Milvus**：向量数据存储与检索服务，支持向量、标量字段、索引、过滤和多种部署形态。

本节选 Milvus，是因为要演示“向量、分块正文、元数据过滤”的完整链路。Milvus Lite 可以从本地单文件起步，后续再讲生产部署。

为什么需要向量库

只有 30 个节点时，用 NumPy 存向量、循环计算余弦相似度就够了。

节点增长到十万或百万后，全量比对的成本会明显增加。

向量库通过**近似最近邻索引**查找与查询最相似的 `k` 条记录，并负责持久化、并发和元数据过滤。

Milvus 的核心概念：Collection

`Collection` 可以理解为一张表，有固定的向量维度、距离度量和标量字段。

代码块

```
store = MilvusVectorStore(
    uri=MILVUS_URI,                    # milvus_login_demo.db, Milvus Lite 单文件
    collection_name="login_demo_chunks",
    dim=1536,
    overwrite=True,
    similarity_metric="COSINE",
)
```

不同模型使用不同集合，避免混入不匹配的向量。

`dim` 要与模型输出维度一致。本次实验设置 `overwrite=True`，会重建集合。

Milvus 的部署形态

根据阶段选择部署方式：

- **Milvus Lite**：开发调试，本地文件，无需单独启动服务。

- **Milvus Standalone**：测试或生产，Docker 或单机部署。
- **分布式集群**：面向更大规模和高可用需求。

本次实验使用 Lite，重点验证检索逻辑，不测试吞吐能力。

对比项	本地文件（Milvus Lite / FAISS）	服务化（Milvus Standalone / 集群）
启动	进程内，一个文件	独立服务，需要运维
并发写入	单进程	支持多客户端
元数据过滤	支持，能力有限	功能更完善，可使用标量索引
适用场景	开发、验证、小规模	生产、团队共用

建库与查库分开

日常查询时，使用 `load_index()` 打开已有索引，不再传入节点。

当前的 `build_index()` 即使设置 `overwrite=False`，仍会提交节点，因此不能当成“只打开数据库”的方法。重复提交并生成新 ID，可能留下重复内容。

库准备好后，下一步看关键词检索怎样工作。

4. BM25

用户说“登出”，资料写“退出登录”，需要语义匹配。

用户直接搜索 `SESSION_TTL_SECONDS`，则可以按这个名字查找代码。

BM25 是经典的关键词检索算法。它为查询词计算权重，主要考虑两点：

- 词在当前文档中出现得越多，通常得分越高。
- 词在整个语料中越少见，区分度越高，权重越大。

“的”“是”“会话”等词到处出现，区分度低，权重也低。`SESSION_TTL_SECONDS` 只在少数位置出现，检索时更容易定位到相关代码。

使用 `rank_bm25` 实现 BM25 检索：

代码块

```
class BM25Retriever:
    def __init__(self, nodes):
        self.nodes = list(nodes)
```

```
        self.corpus = [tokenize(n.text) for n in self.nodes]
        self.bm25 = BM25Okapi(self.corpus)

    def retrieve(self, query, top_k=5):
        scores = self.bm25.get_scores(tokenize(query))
        order = sorted(range(len(scores)), key=lambda i: scores[i], reverse=True)[:top_k]
        return [NodeWithScore(node=self.nodes[i], score=float(scores[i])) for i in order]
```

中文先分词

BM25 按词匹配。中文句子没有空格，需要先分词，否则整句话可能被当作一个词，难以匹配。

本例使用 `jieba`：

代码块

```
def tokenize(text: str) -> List[str]:
    return [t for t in jieba.lcut(text.lower()) if t.strip()]
```

经验：向量检索适合语义表达不同的查询；BM25 适合查找代码标识符等明确关键词。

5. Elasticsearch 的定位与适用场景

BM25 在生产系统中通常由 Elasticsearch 承载。

先区分两者：

- **BM25**：关键词检索的评分方法。
- **Elasticsearch**：提供全文检索与分析能力的系统。

可以用 `rank_bm25` 在进程内运行 BM25，也可以用 Elasticsearch。评分方法相同，工程能力不同。

Elasticsearch 是什么

Elasticsearch 建立在 Lucene 之上，提供倒排索引、BM25 打分，以及分布式分片、副本和聚合分析等能力。

同一索引中的不同字段，可以配置不同的分析器。

本节没有实际部署 Elasticsearch。 这里介绍的是它的定位，不是本次实验结果。

Elasticsearch 与向量库怎么分工

- **精确匹配为主**：日志、审计记录、工单、商品属性等，适合 Elasticsearch。它支持范围查询、聚合统计、字段权重和中文分词。
- **语义相似为主**：用户表达与资料措辞不同，适合向量检索。
- **两种需求都有**：让 Elasticsearch 负责关键词检索，再与向量检索结果融合。

Elasticsearch 7.0 之后支持 `dense_vector` 字段，8.0 之后加入原生 kNN 检索。可以在同一引擎中结合 BM25 与向量检索，也可以用 Milvus 做向量检索、Elasticsearch 做关键词检索，再由应用层融合。

什么时候不需要 Elasticsearch

团队已经用 Elasticsearch 处理日志，可以考虑复用它承载 RAG 的关键词检索。

如果只是一个包含几百份文档的知识库，专门维护 Elasticsearch 集群，运维成本可能高于收益。进程内运行 BM25，或使用 Milvus 的稀疏向量，也能满足这类需求。

结论：Elasticsearch 是工程化的检索与分析引擎，可以承担关键词检索；它不是向量库的替代品。

6. 混合检索（Hybrid Search）：向量 + BM25

向量检索和 BM25 各有强项。混合后是否更好，要用问题集验证。

为什么不能直接相加分数

向量检索返回余弦相似度，通常在 `0` 到 `1` 之间。本批数据中的分数集中在 `0.78` 到 `0.95`，名次之间可能只差 `0.01`。

BM25 分数没有固定上限。查询词没出现时可能为 `0`，出现得多时可以达到十几。

两路打分方式和数值尺度不同，不能直接相加。

用 RRF 融合名次

RRF（Reciprocal Rank Fusion，倒数排名融合）不直接比较两路分数，而是根据名次计算融合分：

代码块

```
def rrf_fuse(result_lists, top_k=5, k=60, weights=None):
    fused, keep = {}, {}
    for weight, results in zip(weights, result_lists):
```

```
            for rank, item in enumerate(results, 1):
                key = item.node.node_id
                fused[key] = fused.get(key, 0.0) + weight / (k + rank)
                keep.setdefault(key, item.node)
        order = sorted(fused.items(), key=lambda kv: kv[1], reverse=True)[:top_k]
        return [NodeWithScore(node=keep[key], score=score) for key, score in order]
```

同一个节点出现在两路结果中，就按节点 ID 累加贡献。

名次越靠前、所在检索路的权重越高，融合分就越高。`k=60` 是 RRF 的融合参数；最终返回数量由 `top_k` 决定。

原 MiniMax 向量 + BM25：混合不保证提升

先比较两路单独检索，再比较三种融合配置：

代码块

```
配置                召回率    命中率    完全命中题数
向量                0.312    0.375    2/8
BM25              0.625    0.75     4/8
RRF等权候选3         0.375    0.5      2/8
RRF等权候选10        0.5      0.625    3/8
RRF加权候选10        0.562    0.625    4/8
```

增加候选数量有帮助，但融合结果仍不如 BM25 单独检索。

本例中，最终候选数量有限，一路的排序可能把另一路的正确证据挤出去。

经验：混合检索不保证提升，必须比较实际结果。

换成 BGE 后：加权融合效果最好

代码块

```
BGE 单独                    召回率=0.688    命中率=0.75     完全命中=5/8
BGE+BM25 等权候选10          召回率=0.75     命中率=0.875    完全命中=5/8
BGE+BM25 等权候选20          召回率=0.75     命中率=0.875    完全命中=5/8
BGE 70% + BM25 30%      召回率=0.812    命中率=0.875    完全命中=6/8
```

本次实验中，`BGE 70% + BM25 30%` 的结果最好。

因此，查询脚本采用这组权重：每路先取 20 条，融合后保留 3 条。

权重和候选数量要通过问题集验证，不能照抄。本次候选从 10 条增加到 20 条，等权结果没有变化。

本节的检索边界

当前 BM25 在进程内运行。以后如果需要服务化的全文检索，可以评估 Elasticsearch。

BM25 是评分方法，Elasticsearch 是检索系统。 本次实验没有部署 Elasticsearch，也没有调用 Milvus 内置的全文检索。

7. 元数据过滤（Metadata Filter）：对象确定了，再排除无关资料

语料里既有登录会话，也有支付会话。

用户查询登录会话时，可以用 `service` 元数据过滤：

```
代码块
1  filtered = index.as_retriever(
2      similarity_top_k=5,
3      filters=service_filter("login_demo"),
4  ).retrieve(q)
```

本次演示返回五条结果。未过滤时，第四条是支付会话；过滤后，这条被排除，前三条保持不变。

结论：本次验证了服务范围过滤，没有证明 Top-3 召回率提高。

过滤范围要根据问题确定：

- 问登录会话：只查登录服务。
- 比较登录与支付：两边资料都要保留。
- 只问“会话”、上下文又没说明服务：先确认查询范围。

当前过滤只是独立演示，还没有接入混合检索。后续接入时，向量检索和 BM25 都要使用相同范围，否则 BM25 仍可能返回其他服务的资料。

8. 检索不准时怎么改进：把用户问题改写成更适合检索的问题

资料范围选对了，仍可能找不到答案。

例如，用户问“用哪条命令验证”，资料标题写的是“测试入口”。

这时可以改写查询，让用户的问法更贴近资料中的表达。

本节继续使用“退出登录”任务，比较查询重写、问题分解、多路改写和术语扩展，并讨论何时需要追问用户。

固定检索条件

先保持检索配置一致：

- 向量模型：BGE
- 关键词检索：BM25
- RRF 权重：`0.7` 和 `0.3`
- 每路候选数：20
- 融合后保留：3 条

运行脚本：

```
代码块
1  python scripts/step3b_query_rewrite.py
```

原问题直接检索的召回率是 `0.812`，八道题中有六道找齐证据。

其中，q4“退出登录改完以后，用哪条命令验证”没有找到标注证据“测试入口”。

接下来比较不同改写方法，看能否补回这条证据。

实验一：查询重写（Query Rewrite）——换成更适合检索的表达

查询重写先让模型整理问题，再用新问题检索。

它可以补充术语，但不能改变用户要查的对象，也不能直接回答问题。

提示词重点看两条：保留原意，只输出查询。

```
代码块
```

```
REWRITE_PROMPT = """把下面的用户问题改写成一句更适合检索的查询。
要求：
- 保留原意，不要回答问题。
- 补上可能使用的正式术语和英文/符号名。
- 去掉不影响含义的口语成分。
- 只输出改写后的查询。

用户问题：{q}
"""
```

这是提示词的精简示意，完整实现见项目中的 `rag/rewrite.py`。

本次 q4 被改写为：

代码块

```
原问题：退出登录改完以后，用哪条命令验证？
改写为：验证退出登录 logout 修改后的命令
```

程序比较了两种方式：

- 只检索改写后的问题
- 原问题和改写问题分别检索，再融合结果

两种方式的召回率都是 `0.812`，完全命中都是 `6/8`。q4 仍未找到“测试入口”。

本次改写改变了表达，没有增加证据覆盖。

单条重写耗时约 `0.8～1.9` 秒。句子更正式，不代表检索效果更好。

保留原问题，可以避免完全丢掉用户的原始表达，但不保证融合结果不变差。最终只保留三条时，新查询会改变排序，原来命中的证据也可能被挤出去。

经验：同时记录原问题和改写问题，检查对象、符号和限制是否保留。

例如，“可能是缓存问题”不能改成“Redis 故障怎么修”，以免把猜测改成确定前提。

实验二：查询分镜（Query Decomposition）——多个需求分别查

问题需要多处资料时，可以拆成子问题，分别检索，再合并结果。

例如，用户问：

> 登录会话和支付会话在有效期、能不能续期上有什么区别？

这里有两类会话、两个比较维度。分解时，四个组合都要覆盖：

- 登录会话的有效期是多少？
- 登录会话是否可以续期？
- 支付会话的有效期是多少？
- 支付会话是否可以续期？

本次模型只输出了前三个子问题，漏掉了“支付会话是否可以续期”。

经验：分解后要对照原问题检查覆盖范围，不能直接使用模型生成的子问题。

本次分解后，召回率仍为 `0.812`，完全命中仍为 `6/8`，没有补回 q4。

这只能说明本次分解没有提高汇总指标。要定位原因，还需查看每个子查询找到的证据，以及合并后保留了哪些证据；不能仅凭汇总结果断定问题出在融合环节。

简单问题也不必强行拆分。用户只问有效期，再增加“定义是什么”“设置在哪里”等子问题，会增加检索工作，也可能带入无关资料。

实验三：多路改写——同一个需求，换几种问法

查询分解把不同需求拆开。

多路改写则针对同一个需求，生成不同问法分别检索。

q4 本次生成了三种问法：

代码块

```
1  退出登录功能验证命令
2  退出登录命令验证代码
3  如何验证退出登录修改后的功能
```

程序分别检索，再融合结果。

脚本标题写着“并行”，但当前代码用循环依次执行，**实际没有并发**。

本次结果

```
原问题:                召回率=0.812，完全命中=6/8
多路改写:              召回率=0.750，完全命中=5/8
多路改写加原问题:      召回率=0.750，完全命中=5/8
```

本次增加问法后，证据覆盖反而下降。 加回原问题，也没有避免下降。

多路查询会累加各路排名分数，但最终仍只保留三条结果，因此可能改变原有证据的排名。要确认具体哪条证据被挤出，需要查看完整候选记录。

经验：是否启用多路改写，要看证据覆盖有没有提升，以及新增调用和检索耗时是否值得。 查询数量增加，不代表检索效果变好。

实验四：术语扩展——保留原问题，补上项目用词

前面几种方法都让模型生成新查询。

这次保留用户原句，再按项目词表补充检索词。

先看“命令”会扩展成哪些项目用词：

```
# 项目词表节选
GLOSSARY = {
    "登出": ["退出登录", "logout"],
    "命令": ["测试入口", "python -m pytest"],
    "多久": ["有效期", "分钟"],
    "改哪些文件": ["需要改的地方"],
}
```

问题中出现“命令”，程序就追加“测试入口”和命令片段。已有词不会重复追加。

这一步按词表扩展，不调用模型生成查询。

q4 扩展为：

```
```

```
退出登录改完以后，用哪条命令验证？ logout 测试入口 python -m pytest
```

原问题保留下来，同时补入了测试指南中的术语。

重新检索后，召回率从 `0.812` 提升到 `0.938`，完全命中题数从 `6/8` 增至 `7/8`。q4 找到了“测试入口”。

各种查询方法对比

方法	召回率	命中率	完全命中
原问题	0.812	0.875	6/8
重写替换原问题	0.812	0.875	6/8
重写与原问题融合	0.812	0.875	6/8
分解	0.812	0.875	6/8
多路改写	0.75	0.875	5/8
多路改写与原问题融合	0.75	0.875	5/8
术语扩展	0.938	1	7/8

命中率 `1.0` 表示八道题都至少找到一条标注证据。

它不表示八道题都找齐了证据，也不表示最终答案全部正确。

第一名不变，整组结果仍可能改善

q4 扩展前后，排名第一的都是需求记录中的“验收标准”。

但召回率从 `0` 变成 `1`。根据这道题的 gold，“测试入口”进入了第二或第三位。

经验：检查召回要看全部三条结果，不能只看第一名。

本次结果能说明：追加这组项目术语后，q4 的正确证据进入了最终结果。

由于没有两路检索各自的详细候选记录，不能进一步断定向量检索和 BM25 都命中了这条证据。

这份词表针对当前项目和问题集调整。后续还要用新问题验证，并根据失败查询更新词表，不能把这次提升直接推广到所有项目。

9. 查询澄清：如果对象没说清，先确定对象

查询改写解决不了对象不明确的问题。

例如，用户只问“会话多久过期”，资料中同时有登录会话和支付会话。

如果当前上下文不能确定是哪种会话，就先确认对象，再按明确范围检索。

本次脚本输出：

```
代码块
1  会话多久过期?
2  → 需要澄清：请问是哪个服务的会话?
3
4  登录会话的有效期是多少分钟?
5  → 明确
6
7  SESSION_TTL_SECONDS 是多少?
8  → 需要澄清：请明确是 login_demo 还是 pay_gateway 服务?
```

前两条说明：对象是否明确，会影响能否直接检索。

第三条还要先查符号。如果 `SESSION_TTL_SECONDS` 在当前项目范围内只指向一个对象，可以直接检索；如果多个服务都用了这个名字，再追问用户。

经验：不能只凭模型说“需要澄清”，就打断用户。 先检查现有上下文和符号，确认确实无法确定对象后再追问。

查错对象的风险也不只是回答含糊，还可能让用户采用另一个服务的规则。

当前脚本只打印澄清判断，没有接收用户补充后再次检索。因此，这只是独立演示，还不是完整的追问流程。

这一节的方法如何用于项目

先判断证据遗漏的原因，再选择处理方式：

- 对象不明确：先澄清。
- 对象明确，但用词与资料不同：尝试查询重写或术语扩展。
- 问题包含多个信息需求：分解后检查是否覆盖完整。
- 同一需求有多种检索表达：尝试多路改写，并检查最终结果是否改善。

处理后的查询必须重新查库。

当前 `hybrid()` 会针对每个查询重新执行向量和 BM25 检索，再融合。如果只在旧候选中重排，就找不到原先未进入候选的证据。

本批问题中，术语扩展提高了召回，多路改写反而降低了召回。下一步要用独立问题集，同时记录质量、成本和延迟，再决定哪些方法进入正式流程。

关键证据进入前三，只表示模型拿到了资料。 接下来还要检查证据怎样筛选、压缩，以及回答是否与证据一致。

第四步：检索后处理（解决结果质量问题）——选好证据，再检查回答

第三步调整检索和查询，让更多正确证据进入结果。

接下来要解决三个问题：

- 候选中已有正确资料，怎样把它排到前面？
- 怎样压缩证据，减少无关内容？
- 怎样检查模型是否正确使用了证据？

rerank 的核心机制是：把“用户问题”和“每个候选片段”放在一起判断相关性，再按分数重新排序。

1. 数据进入了候选

继续围绕退出登录任务。用户问“用哪条命令验证”，正确证据是测试指南中的“测试入口”。

如果正确证据没有进入候选，重排就无法选中它。运行补充脚本，对比候选深度：

代码块

```
python scripts/step4b_rerank_limit.py
```

本次它进入了 20 条候选，却排在最后。直接取前三，仍然看不到它。

代码块

```
q4 候选20召回=1.0  候选20命中=1.0  候选3命中=0.0
   gold 所在位置: [20]
```

“测试入口”已经进入 20 条候选，但前三没有。

因此，可以尝试重排：让程序重新判断哪些候选能回答“用哪条命令验证”。

2. Rerank：重新判断哪条证据能回答问题

向量检索找语义相近的内容，BM25 找关键词匹配的内容。

但“话题相关”不等于“能直接回答问题”。

例如，“退出登录验收标准”与用户问题有关，却不一定包含测试命令。

重排（Rerank）：在已有候选中，再按“能否回答当前问题”重新排序。

LLM 重排：让模型返回候选顺序

提示词要求模型排序，不生成答案；返回编号，方便程序取回原文。

代码块

```
RERANK_PROMPT = """你在给检索结果重排。下面是用户问题和 {n} 条候选证据。
按“这条证据能直接回答用户问题”的程度从高到低排序。
只输出一个 JSON 数组，元素是候选编号（从 1 开始）。
不要解释，不要代码块；无关证据放在末尾。

用户问题：{q}

候选证据：
{items}
"""
```

Cross Encoder：对问题和候选逐条打分

Cross Encoder（交叉编码器）把问题和一条候选一起输入模型，得到相关性分数，再按分数排序。

它需要逐条评估候选，通常放在初步检索之后。本课将同一批 20 条候选交给本地重排模型：

代码块

```
python scripts/step4c_cross_encoder.py
```

核心调用关注两个输入：原问题和候选正文。

代码块

```
from fastembed.rerank.cross_encoder import TextCrossEncoder

ce = TextCrossEncoder("BAAI/bge-reranker-base")
scores = list(ce.rerank(question, [h.node.text for h in hits]))
```

程序按返回分数取前三。

分数用于排序，不代表答案正确的概率。

本次重排结果

方法	召回率@3	命中率@3	完全命中
直接取前三	0.812	0.875	6/8
LLM 重排	0.812	0.875	6/8
Cross Encoder 重排	0.875	0.875	7/8

输入：把问题和一个候选片段组成一对，一起送入模型。

判断：模型通过注意力机制，让问题中的词与片段中的词相互关联，判断片段是否符合问题要找的信息。

输出：为每个候选片段给出相关性分数，再选排名靠前的片段交给回答模型。

它与向量检索的关键区别是：**向量检索分别编码问题和片段，再比较向量；Cross-Encoder 联合阅读问题和片段，直接判断两者的关系。** 因此通常判断更细，但计算更慢，适合处理检索已经找回的小批候选。

3. 上下文压缩：少给无关内容，保住规则条件

取出前三条后，证据里仍可能有大量代码和背景。

例如，用户只问常量值，却拿到整份源码。

压缩的目标：保留回答需要的内容，减少无关文本。

当前实现让模型从每段证据中摘录相关句子。

看提示词怎样约束压缩

代码块

```
COMPRESS_PROMPT = """从下面这段证据里，只摘出能回答用户问题的句子。
要求：
- 逐字摘录，不要改写，不要补充原文没有的内容。
- 保留小标题和代码符号。
- 无关时只输出：无相关内容。

用户问题：{q}

证据：
{text}
"""
```

提示词要求逐字摘录、不补充内容。

如果模型输出“无相关内容”或文本少于 8 个字符，当前程序会丢弃这段证据；其余内容保留来源元数据，供后续引用。

检查压缩结果是否可靠

两个规则都需要核对：

- 短于 8 个字符，不代表一定没有用。很短的常量定义也可能是有效证据。
- 要求逐字摘录，不代表模型一定没有改写。

q6 的本次结果：

代码块

```
压缩前3022字符，压缩后保留2段、136字符。
```

上下文明显变短，但这道题压缩后，无依据实体从 1 个增加到 2 个。

文本更短，不代表回答更可靠。

当前回执没有展示完整摘录和答案，因此还不能确定问题来自删漏、改写，还是最终回答。

保留适用条件与版本

压缩时要检查范围、版本、否定和条件是否保留。

例如，原文是“旧版本使用 `<=`，导致恰好到期仍有效”。如果删掉“旧版本”，读者可能误以为当前实现仍有这个问题。

资料冲突时，先核对适用范围和版本；无法确认，就明确说明冲突，不要合并成一条新规则。

这项冲突检查是后续需要增加的能力，当前脚本没有自动处理。

4. 三种处理一起看：收益有没有抵过代价

主脚本比较三条链路：

- **A：检索直出**：取前三条，生成回答。
- **B：增加重排**：重排后取前三，再生成回答。
- **C：增加压缩**：沿用重排结果，压缩证据后生成回答。

本次汇总：

配置	必备/禁用事实检查通过	引用合格率（脚本标签）	无依据实体数	计入耗时	最终回答输入 Token
A 检索直出	8/8	8/8	4	32.4秒	7656
B 加重排	8/8	8/8	1	62.0秒	7721
C 加重排、压缩	8/8	8/8	3	93.5秒	5926

先看 8/8：脚本检查范围有限

表中的 8/8 被脚本标为答案正确，但实际检查的是：

- `required_facts` 是否出现
- `forbidden_facts` 是否没有出现

例如，q3 只要求出现 `app/auth.py`，没有检查是否列出全部修改文件；q7 要求出现 30 分钟和 5 分钟，没有完整检查续期情况。

所以，8/8 不能证明回答完整、正确。

再看无依据实体：指标可能一升一降

本次重排后，无依据实体从 4 个降到 1 个；压缩后又升到 3 个。

不能只看上下文是否变短，要同时检查回答依据和覆盖情况。

最后看耗时与 Token

压缩后，最终回答输入从 7721 Token 降到 5926 Token，减少约 23%。

但计入耗时从 62.0 秒增加到 93.5 秒。

原因是当前压缩会逐段调用模型：最终回答的输入变短了，但增加了压缩请求。

表中没有统计重排和压缩请求的 Token，因此不能据此说总费用降低了 23%。耗时也不包括建库、候选检索和校验，只统计各链路计入的生成与后处理时间。

结论：本次实验没有证明某条链路在质量、费用和速度上全面更好。

决定是否启用重排或压缩前，还要统计完整调用的 Token 与延迟，并用覆盖更完整的问题标注来评估质量。

5. Citation：编号合法，只是引用检查的第一步

先检查回答中的 `[1]`、`[2]` 能否对应到本轮提供的证据，再检查**证据是否支持引用它的那句话**。

本次三条链路的引用编号检查都是 8/8，但 q4 中，A 和 C 仍各有一个在证据中找不到的实体，B 没有。说明**编号合法、必备关键词出现，都不能证明整段回答有依据**。

当前日志没有展示具体答案和实体，需要回看完整记录，才能定位哪句话缺少依据。

6. 一致性校验：检查覆盖了什么，也说明没覆盖什么

这里的一致性校验，是当前代码增加的检查逻辑。 我们可以借此来修改和完善：提取哪些内容、怎样核对、发现问题后怎么处理。

为什么需要这项检查

模型拿到了资料，仍可能编造信息，或者把资料中的条件说错。引用编号正确，也不能保证回答正确。

它引用了资料，却改变了起算条件和时间单位。因此，我们需要在生成回答后，检查回答是否忠于证据。

当前代码怎样检查

当前代码先做基础核对：提取回答中的文件路径、常量名、数字与单位、命令、布尔值，再检查它们能否在证据中找到。

这能发现没有来源的路径、数字或命令，但不能完整理解句意。比如前面的登录案例，**仅检查数字是否出现，发现不了起算条件被改了。**

可以进一步优化：把“回答中的关键结论”和“对应原文”交给校验模型，要求它判断证据是否支持结论，并指出被改动的条件、范围或单位。模型校验仍可能出错，关键结论需要规则或人工复核。

如果回答做了换算，例如把“30 分钟”写成“1800 秒”，应核对计算过程；如果证据经过压缩，还应回到原文检查，避免压缩时就改错了内容。

检查出问题以后怎么办

当前脚本只打印结果，不会自动修正，也不会拦截回答。 接入产品时，可以增加处理流程：

- **有证据，但回答说错了**：把错误位置和原文交给模型，要求重新生成，再检查。
- **证据不足**：补充检索；仍找不到依据，就明确说明无法确认。
- **多次检查仍未通过**：停止自动重试，转人工复核或返回无法确认的提示。

如果要求用户只看到检查通过的回答，就采用“生成 → 校验 → 展示”的顺序。直接展示流式内容，事后检查无法阻止用户看到已经输出的错误信息。

事后校验不能阻止用户看到已经展示的内容。 发布方式要结合交互需求设计。

接下来：验证整条链路

重排负责重新选择证据，压缩负责减少无关内容，引用与校验负责发现回答中的问题。

本次重排改善了部分指标；压缩减少了最终输入，但增加了耗时，也增加了未通过实体检查的数量。因此，暂时把它们作为候选方案比较。

下一步补齐每道题的正确证据和回答要求，统一比较**召回率、答案完整性、证据支持情况、全部调用成本和延迟**，再决定采用哪条链路。

第五步：验证效果

前面调整了检索、重排和压缩。

现在用同一组问题检查三件事：

- 资料找得更准了吗？
- 回答符合要求了吗？
- 增加的时间和调用是否值得？

本节先运行脚本，再看总览和具体题目，最后比较配置。

```
代码块
1   python scripts/step5_evaluate.py
```

先认识四组配置

配置	做什么
V0	MiniMax向量检索，直接取前三回答
V1	术语扩展，BGE与BM25检索，加权融合后取前三回答
V2	使用扩展查询召回二十条候选，LLM重排取前三回答
V3	复用V2的重排结果，压缩后回答

四组配置使用相同的 9 份资料、30 个分块和 8 道题。

分块方式和最终回答提示词保持不变，比较对象是四组配置的实际效果。

1. 先看总览：哪些改动值得继续检查

先看质量相关的五列，再读具体题目。

配置	召回率@3	命中率@3	答案检查通过	引用检查通过	含无依据实体的回答
V0	0.312	0.375	4/8	6/8	2/8
V1	0.938	1	8/8	6/8	1/8
V2	0.812	0.875	8/8	5/8	2/8
V3	0.812	0.875	8/8	6/8	1/8

总览中有三个现象：

- **V0 → V1**：找回了更多标注证据；答案检查从 4 题通过变为 8 题通过。本组问题中，检索优化有效。
- **V1 → V2**：加入重排后，召回率反而下降。多一个处理环节，不保证结果更好。
- **V2 → V3**：答案检查仍为 `8/8`，引用检查有所改善。但还要结合 Token 用量和耗时，判断是否值得开启压缩。

接下来先理解指标，再看逐题结果。

2. 用一道题理解召回率与命中率

V1 的逐题结果是：

```
代码块
1    q2：召回率=0.5，答案检查=√，引用判定=无可核对实体
```

因此，V1 可以同时出现命中率 `1.0` 和召回率 `0.938`：八道题都至少找到一处证据，但 q2 只找到了一半。

命中率看“有没有找到”，召回率看“找回了多少”。两者不能互相替代。

3. 三个判断分开看

逐题输出中，每格依次显示：

召回率 / 答案检查 / 引用判定

q2：答案检查通过，但没有找到标注证据

V0 的结果是：

```
代码块
1   q2: 0.0 √ 通过
```

这行结果说明需要复核回答依据，不能直接读成“系统完全正确”或“模型一定答错”。

q4：重排后，测试入口没有留在结果里

q4 问：“退出登录改完以后，用哪条命令验证？”

`gold` 是测试指南中的“测试入口”。

```
代码块
1   V0: 0.0 × 通过
2   V1: 1.0 √ 通过
3   V2: 0.0 √ 通过
4   V3: 0.0 √ 通过
```

逐行来看：

- **V0**：没找到“测试入口”，回答也没出现必备命令；引用检查仍然通过。说明“引用通过”不等于回答完整。
- **V1**：找到了“测试入口”，回答满足关键词要求。
- **V2、V3**：结果中没有“测试入口”，回答却出现了必备命令。它可能来自其他有效证据，也可能有其他来源，需要查看完整回答和证据才能判断。

下一步检查候选和重排输入：测试入口是否进入候选、重排实际看到了哪些文本、最终为什么没有选中它。不能只凭最终结果就把原因归到初始检索。

还要确认是否有其他证据支持答案。如果有，应补全可接受证据标注；如果没有，关键词检查通过也不能代替证据核对。

q6：标注来源命中了，实体检查仍未通过

q6 问 `SESSION_TTL_SECONDS` 的值。V1、V2、V3 都是：

```
代码块
```

```
q6：1.0 √ 存在无依据实体
```

来源命中不代表引用正确；标准数字出现，也不代表整段解释都有依据。

4. 再看汇总：哪些数字是怎么来的

读过三道题后，再看总览数字，就能知道它们代表什么。

- **V0 答案检查：4/8**
  q1、q2、q7、q8 通过，其余四题未通过。
  归因表只把 q1、q7、q8 标为“通过”，因为 q2 没有命中 `gold`。
- **V1 引用检查：6/8**
  q2 是“无可核对实体”，q6 是“存在无依据实体”。两题都不计为通过。
- **V2 引用检查：5/8**
  在 V1 基础上，q3 也出现了无依据实体。
- **V3 引用检查：6/8**
  q3 恢复通过。

逐题变化如何影响召回率

V1 到 V2：

- q4 从 `1` 降到 `0`。
- q2 从 `0.5` 升到 `1`。
- q7 从 `1` 降到 `0.5`。

经验：平均数发生变化时，回到逐题结果，确认具体是哪几题变了。

5. “正确率”与“失败归因”要按代码理解

脚本把一列命名为“答案正确率”，实际执行的是关键词检查：

- 必备事实都出现
- 禁用事实都没有出现

满足这些条件，就记为通过。

归因标签也由特定规则决定：

```
代码块
1   if f["correct"] and gold_in:
2       failure = "通过"
3   elif not gold_in:
4       failure = "检索失败"
5   else:
6       failure = "生成失败"
```

先确认原文、解析和分块正确，再针对发现的问题调整对应环节。一次只改一类因素，才能看清改动效果。

6. 质量之外，怎样读时间与用量

先看本次记录：

配置	最终回答输入Token总量	最终回答输出Token总量	记录的总耗时	记录的平均耗时
V0	6598	1195	52.0秒	6.5秒
V1	7180	592	23.7秒	3.0秒
V2	8440	904	66.7秒	8.3秒
V3	6132	777	66.3秒	8.3秒

压缩减少输入，不等于总费用下降

相同耗时，计时范围可能不同

建库时间与在线时间分开统计

```
代码块
1   [建库] V0: 6.0秒; V1–V3的BGE索引: 2.8秒
```

```
2   MiniMax Embedding: 14次请求、38条文本、9.1秒
```

费用也要分开统计：

- **资料准备**：读取、解析、分块、建库和索引更新。
- **在线查询**：查询、重排、压缩和回答。

本地模型没有远程 API 费用，但仍会占用计算资源。

经验：用量缺失时标为“未知”，不要填成 0。 当前脚本把缺失的 `usage` 填为 0；正式报告应修改这一处理。没有实际用量和单价，就不能判断费用。

7. 这次结果支持什么选择

因此，**V1 可以先作为候选默认方案**，再用新问题验证。

V2 没有提高答案检查通过率，召回和引用检查反而下降。

V3 相对 V2 减少了最终输入，并改善了一项引用结果；但完整费用和端到端延迟尚未统计，暂时不能据此决定启用压缩。

评估要回答三个问题：哪个环节改善了、哪些问题还没解决、下一步具体改什么。

小结

本节围绕登录会话项目和八道题，跑通 RAG 流程，并逐步检查效果。

第一步：跑通链路

从读取资料、分块、向量化和入库，到检索、装配、回答和校验，逐步保留输出。

答错时，检查模型实际拿到了哪些资料，不只根据最终答案猜原因。

第二步：准备好资料

解析要保留条件与结果的对应关系。

清洗不能删掉否定词和适用范围。

分块要保留完整规则和代码结构；元数据要记录来源，供后续过滤和引用。

第三步：改进召回

在这批问题上，更换 BGE、加入 BM25 和加权融合，再补充项目术语后，召回率从 `0.312` 提高到 `0.938`。

多路改写的召回率则出现下降。

每项改动都要实测，不能因为方法更复杂，就认为效果更好。

第四步：处理候选并检查回答

重排从已有候选中重新选择证据。

压缩减少无关内容，但可能丢失条件，也会增加调用。

引用帮助回到原文；校验帮助检查回答是否有依据。

重排无法补回候选之外的资料。

第五步：按统一口径比较

分别检查证据是否找到、答案是否完整、引用是否支持结论。

定位问题时，依次确认：

1. 候选中有没有正确证据？
2. 最终输入中有没有这条证据？
3. 模型有没有正确使用它？

成本要统计完整调用，延迟要使用相同的计时范围，结果才有可比性。

附录：配套代码怎么跑

代码块

```
# 目录
corpus/                 语料：docs/(md) app/(py) history/(md) web/(html)
tables/(csv) pdf/
eval/questions.json     8 道问题，每题带权威出处、必备事实、禁止事实
rag/                    公共库：解析入库、检索、查询改写、装配校验、指标口径
scripts/                按步骤分开的七个脚本 + 三个补充实验脚本
runs/                   每次运行的标准输出
```

```
执行代码
python scripts/step1_min_loop.py          # 第一步：最小闭环
python scripts/step2_data_prep.py         # 第二步：解析、清洗、六种分块对比
python scripts/step3_retrieval.py         # 第三步：向量/BM25/Hybrid/过滤/模型与维度
python scripts/step3b_query_rewrite.py    # 第三步补充：重写、分解、澄清、术语扩展
python scripts/step4_postprocess.py       # 第四步：重排、压缩、引用、一致性校验
python scripts/step4b_rerank_limit.py     # 第四步补充：重排的边界
python scripts/step5_evaluate.py          # 第五步：四种配置端到端评估

# 第四步补充：Cross Encoder 重排，和 LLM 重排对比
python scripts/step4c_cross_encoder.py    # 首次运行会下载bge-reranker-base（约1GB）

# Milvus 的两个坑要分三个进程跑，输出拼成一个文件
python scripts/extra_milvus_pitfalls.py build
python scripts/extra_milvus_pitfalls.py again
python scripts/extra_milvus_pitfalls.py noload

# 补充技术点：空块、稳定 ID、资料内注入、无答案题、元数据、分词、Embedding 稳定性
python scripts/extra_technical_checks.py
```